

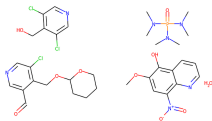
Logical Aspects of Graph Neural Networks

Michael Benedikt¹ Tony Tan²

¹University of Oxford

²University of Liverpool

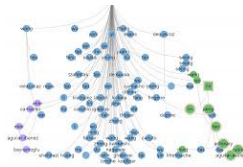
Graphs are everywhere



molecules



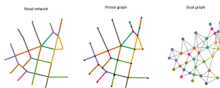
social networks



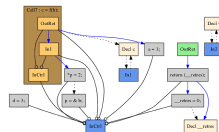
citation networks



knowledge graphs



road networks



program dependence

Why graphs?

Most data is *relational*:

Why graphs?

Most data is *relational*:

Domain	Nodes / edges	Task
Chemistry	atoms / bonds	is this molecule toxic?
Finance	accounts / transactions	is this account laundering money?
Road networks	segments / junctions	what is the travel time?
Social platforms	users / follows	is this account a bot?
Knowledge graphs	entities / relations	is this fact missing?
Software	statements / control flow	is there a bug?

Why graphs?

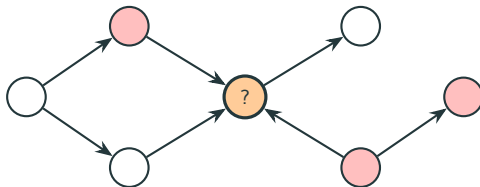
Most data is *relational*:

Domain	Nodes / edges	Task
Chemistry	atoms / bonds	is this molecule toxic?
Finance	accounts / transactions	is this account laundering money?
Road networks	segments / junctions	what is the travel time?
Social platforms	users / follows	is this account a bot?
Knowledge graphs	entities / relations	is this fact missing?
Software	statements / control flow	is there a bug?

GNNs are now the prevalent model for learning such functions.

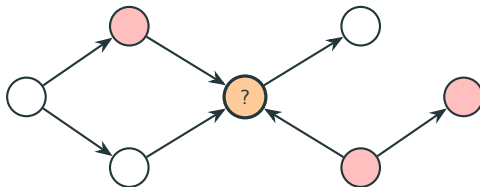
What we ask a GNN to compute

(Node classification) Given a graph and a node, output a label.



What we ask a GNN to compute

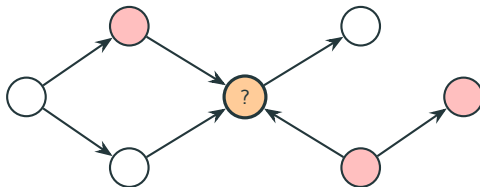
(Node classification) Given a graph and a node, output a label.



(Graph classification) Given a graph, output one label for the whole graph.
(Is this molecule soluble? Is this program malicious?)

What we ask a GNN to compute

(Node classification) Given a graph and a node, output a label.



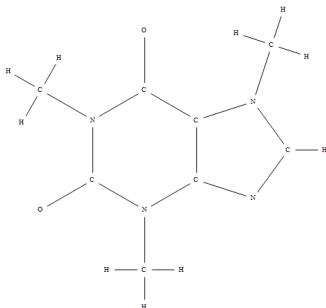
(Graph classification) Given a graph, output one label for the whole graph.
(Is this molecule soluble? Is this program malicious?)

This talk:

- GNN verification with expressive logic.
- GNN analysis with convergence law from logic. (in 2nd part by M. Benedikt)

Setting: Graphs with finitely many colors on vertices

Example: Molecules as graphs



(Vertices) atoms

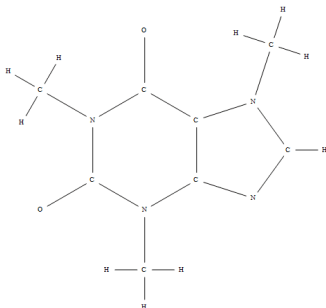
(Edges) bonds

(Colors) the element: U_C , U_N , U_O , U_H

Each vertex carries a $\{0,1\}$ -vector recording which colors hold — its *initial feature*.

Setting: Graphs with finitely many colors on vertices

Example: Molecules as graphs



(Vertices) atoms

(Edges) bonds

(Colors) the element: U_C , U_N , U_O , U_H

Each vertex carries a $\{0,1\}$ -vector recording which colors hold — its *initial feature*.

Interesting chemical properties involve *counting*:

- A carbon with three hydrogens.
- More oxygens than nitrogens among the neighbours.

Why verify?

Training optimises average loss over available sample, but does not guarantee that the model is non-trivial.

Why verify?

Training optimises average loss over available sample, but does not guarantee that the model is non-trivial.

(Eg.) A GNN that detects suspicious accounts may achieve 99% accuracy on the test set. But it turns out that it is a trivial classifier that never flags any account as suspicious.

The formal questions

We focus on binary classifiers: GNNs that either accept or reject a node.

(Satisfiability) Given a GNN $\mathcal{A}$, is there a graph G and node v with $\mathcal{A}$ accepting $\langle G, v \rangle$?

(Universal satisfiability) Given a GNN $\mathcal{A}$, is there a graph G such that $\mathcal{A}$ accepts $\langle G, v \rangle$ for every node v in G ?

The formal questions

We focus on binary classifiers: GNNs that either accept or reject a node.

(Satisfiability) Given a GNN $\mathcal{A}$, is there a graph G and node v with $\mathcal{A}$ accepting $\langle G, v \rangle$?

(Universal satisfiability) Given a GNN $\mathcal{A}$, is there a graph G such that $\mathcal{A}$ accepts $\langle G, v \rangle$ for every node v in G ?

Fact

For unrestricted GNNs, these problems are undecidable.

The formal questions

We focus on binary classifiers: GNNs that either accept or reject a node.

(Satisfiability) Given a GNN $\mathcal{A}$, is there a graph G and node v with $\mathcal{A}$ accepting $\langle G, v \rangle$?

(Universal satisfiability) Given a GNN $\mathcal{A}$, is there a graph G such that $\mathcal{A}$ accepts $\langle G, v \rangle$ for every node v in G ?

Fact

For unrestricted GNNs, these problems are undecidable.

Strategy:

- Fix a restricted class of GNNs.
- Find the logical characterisation of the class.
- Use the toolkit from the logic to reason about them.

The logic also gives us *expressiveness* results.

Outline of this tutorial

1. A brief review of GNNs
2. GNN and the WL algorithm
3. Uniform expressiveness: the logic C^2
4. Presburger quantifiers and the logic MP^2
5. GNNs with bounded activations: exact characterisation
6. GNNs with unbounded activations
7. Summary

Table of contents

1. A brief review of GNNs
2. GNN and the WL algorithm
3. Uniform expressiveness: the logic C^2
4. Presburger quantifiers and the logic MP^2
5. GNNs with bounded activations: exact characterisation
6. GNNs with unbounded activations
7. Summary

The formal model – colored graphs

(Def.) A *colored graph signature* has one binary predicate E and unary predicates $U_1, \dots, U_n$ (the *colors*).

An n -graph is $G = \langle V, E, U_1, \dots, U_n \rangle$, with V finite and nonempty, $E \subseteq V \times V$.

Formal definition of GNNs

An L layer GNN $\mathcal{A}$: $\left\langle f^{(\ell)}, \text{COMBINE}^{(\ell)}, \text{AGG}^{(\ell)}, \text{READOUT}^{(\ell)} \right\rangle_{\ell=1}^L$.

Input: An n -color graph G where the *initial feature* of a node v is the one-hot encoding of its colors: $\xi_i^{(0)}(v) = 1$ iff $v \in U_i$.

Formal definition of GNNs

An L layer GNN $\mathcal{A}$: $\left\langle f^{(\ell)}, \text{COMBINE}^{(\ell)}, \text{AGG}^{(\ell)}, \text{READOUT}^{(\ell)} \right\rangle_{\ell=1}^L$.

Input: An n -color graph G where the *initial feature* of a node v is the one-hot encoding of its colors: $\xi_i^{(0)}(v) = 1$ iff $v \in U_i$.

On level ℓ , it updates the feature vector of each node:

$$\bar{u} \mapsto f(\text{COMBINE}(\bar{u}, \text{AGG}\{\text{features of the neighbors}\}, \text{READOUT}\{\text{features of all nodes}\}))$$

Formal definition of GNNs

An L layer GNN $\mathcal{A}$: $\left\langle f^{(\ell)}, \text{COMBINE}^{(\ell)}, \text{AGG}^{(\ell)}, \text{READOUT}^{(\ell)} \right\rangle_{\ell=1}^L$.

Input: An n -color graph G where the *initial feature* of a node v is the one-hot encoding of its colors: $\xi_i^{(0)}(v) = 1$ iff $v \in U_i$.

On level ℓ , it updates the feature vector of each node:

$$\bar{u} \mapsto f(\text{COMBINE}(\bar{u}, \text{AGG}\{\text{features of the neighbors}\}, \text{READOUT}\{\text{features of all nodes}\}))$$

For directed graphs:

$$\bar{u} \mapsto f(\text{COMBINE}(\bar{u}, \text{AGG}_{in}\{\text{features of the in-neighbors}\}, \text{AGG}_{out}\{\text{features of the out-neighbors}\}, \text{READOUT}\{\text{features of all nodes}\}))$$

COMBINE = aff. trans., AGG = sum, READOUT = sum / 0.

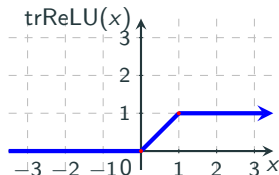
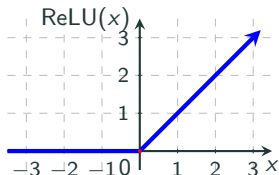
Formal definition of GNNs – Cont'd

An L layer GNN $\mathcal{A}$: $\left\langle f^{(\ell)}, \text{COMBINE}^{(\ell)}, \text{AGG}^{(\ell)}, \text{READOUT}^{(\ell)} \right\rangle_{\ell=1}^L$.

Formal definition of GNNs – Cont'd

An L layer GNN $\mathcal{A}$: $\left\langle f^{(\ell)}, \text{COMBINE}^{(\ell)}, \text{AGG}^{(\ell)}, \text{READOUT}^{(\ell)} \right\rangle_{\ell=1}^L$.

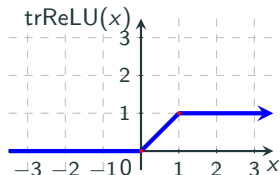
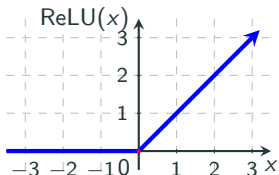
The activation function $f = \text{ReLU}$ / truncated ReLU.



Formal definition of GNNs – Cont'd

An L layer GNN $\mathcal{A}$: $\left\langle f^{(\ell)}, \text{COMBINE}^{(\ell)}, \text{AGG}^{(\ell)}, \text{READOUT}^{(\ell)} \right\rangle_{\ell=1}^L$.

The activation function $f = \text{ReLU}$ / truncated ReLU.

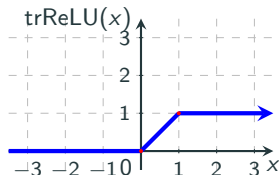
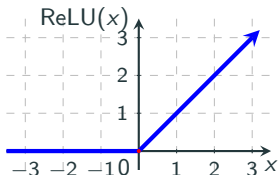


Can be easily generalised to: *piecewise linear functions* and *eventually constant functions*.

Formal definition of GNNs – Cont'd

An L layer GNN $\mathcal{A}$: $\left\langle f^{(\ell)}, \text{COMBINE}^{(\ell)}, \text{AGG}^{(\ell)}, \text{READOUT}^{(\ell)} \right\rangle_{\ell=1}^L$.

The activation function $f = \text{ReLU}$ / truncated ReLU .



Can be easily generalised to: *piecewise linear functions* and *eventually constant functions*.

(Def.) We consider GNN $\mathcal{A}$ where in the layer L , the feature of each node has dimension 1.

$\mathcal{A}$ accepts $\langle G, u \rangle$ if $\xi^{(L)}(u) \geq .5$.

GNN training and verification

GNN training algorithm: (PyTorch, JAX, TensorFlow, etc.)

Input: Training data D

Output: A GNN model $\mathcal{A}$ (i.e., the matrices for COMBINE)

GNN training and verification

GNN training algorithm: (PyTorch, JAX, TensorFlow, etc.)

Input: Training data D

Output: A GNN model $\mathcal{A}$ (i.e., the matrices for COMBINE)

(Q1) For what task can we train GNN? (i.e., what is the expressiveness of GNNs?)

(Q2) How can we verify the output GNN model $\mathcal{A}$? (e.g., is $\mathcal{A}$ a trivial classifier?)

GNN training and verification

GNN training algorithm: (PyTorch, JAX, TensorFlow, etc.)

Input: Training data D

Output: A GNN model $\mathcal{A}$ (i.e., the matrices for COMBINE)

(Q1) For what task can we train GNN? (i.e., what is the expressiveness of GNNs?)

(Q2) How can we verify the output GNN model $\mathcal{A}$? (e.g., is $\mathcal{A}$ a trivial classifier?)

Tool: Comparison with known graph algorithms (WL algorithm) and application of tools from logic.

Two verification problems

Due to undecidability of general GNN, we will fix some class $\mathcal{C}$ of GNNs.

(Satisfiability) Given $\mathcal{A} \in \mathcal{C}$, is there an n -graph G and a node v in G such that $\mathcal{A}$ accepts $\langle G, v \rangle$?

Two verification problems

Due to undecidability of general GNN, we will fix some class $\mathcal{C}$ of GNNs.

(Satisfiability) Given $\mathcal{A} \in \mathcal{C}$, is there an n -graph G and a node v in G such that $\mathcal{A}$ accepts $\langle G, v \rangle$?

(Universal satisfiability) Given $\mathcal{A} \in \mathcal{C}$, is there an n -graph G such that $\mathcal{A}$ accepts $\langle G, v \rangle$ for every node v of G ?

Table of contents

1. A brief review of GNNs
2. GNN and the WL algorithm
3. Uniform expressiveness: the logic C^2
4. Presburger quantifiers and the logic MP^2
5. GNNs with bounded activations: exact characterisation
6. GNNs with unbounded activations
7. Summary

Weisfeiler-Leman (WL) algorithm

Introduced in 1968 as an algorithm to detect graph isomorphism:

Input: Two graphs G and H

Task: Decide whether they are isomorphic

Weisfeiler-Leman (WL) algorithm

Introduced in 1968 as an algorithm to detect graph isomorphism:

Input: Two graphs G and H

Task: Decide whether they are isomorphic

(WL algorithm) Run the following on the disjoint union of G and H .

Start with the trivial partition $\{V\}$.

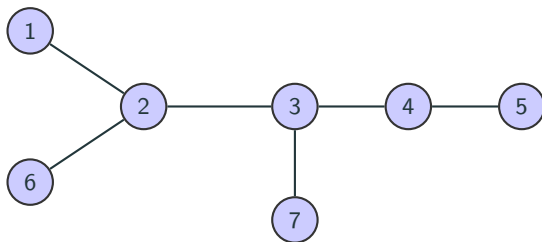
Iterate until the partition stops refining:

- for each node, count its neighbours in each block;
- split every block according to these counts.

Output “YES” if every block holds equally many vertices from G and of H .

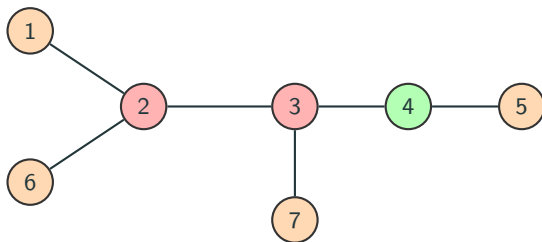
Output “NO” otherwise, i.e., WL cannot tell G and H apart.

WL algorithm – An example on one graph



$$P_0 = \{ \{1, 2, 3, 4, 5, 6, 7\} \}$$

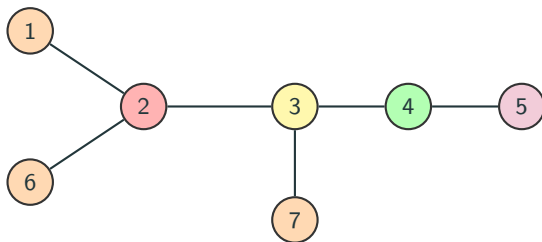
WL algorithm – An example on one graph



$$P_0 = \{ \{1, 2, 3, 4, 5, 6, 7\} \}$$

$$P_1 = \{ \{1, 5, 6, 7\} \{2, 3\} \{4\} \}$$

WL algorithm – An example on one graph

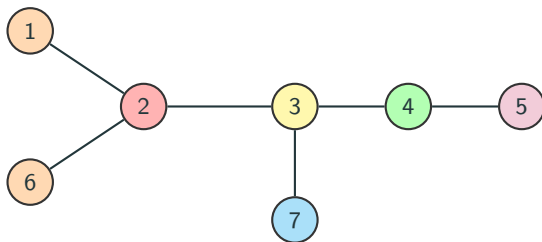


$$P_0 = \{ \{1, 2, 3, 4, 5, 6, 7\} \}$$

$$P_1 = \{ \{1, 5, 6, 7\} \{2, 3\} \{4\} \}$$

$$P_2 = \{ \{1, 6, 7\} \{5\} \{2\} \{3\} \{4\} \}$$

WL algorithm – An example on one graph



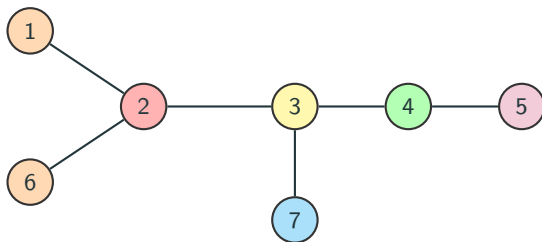
$$P_0 = \{ \{1, 2, 3, 4, 5, 6, 7\} \}$$

$$P_1 = \{ \{1, 5, 6, 7\} \{2, 3\} \{4\} \}$$

$$P_2 = \{ \{1, 6, 7\} \{5\} \{2\} \{3\} \{4\} \}$$

$$P_3 = \{ \{1, 6\} \{7\} \{5\} \{2\} \{3\} \{4\} \}$$

WL algorithm – An example on one graph



$$P_0 = \{ \{1, 2, 3, 4, 5, 6, 7\} \}$$

$$P_1 = \{ \{1, 5, 6, 7\} \{2, 3\} \{4\} \}$$

$$P_2 = \{ \{1, 6, 7\} \{5\} \{2\} \{3\} \{4\} \}$$

$$P_3 = \{ \{1, 6\} \{7\} \{5\} \{2\} \{3\} \{4\} \}$$

After this step, the partition does not change.

Weisfeiler-Leman (WL) algorithm – cont'd

(WL algorithm)

Output “YES” if G and H have the same count of each color.

Output “NO” otherwise.

Weisfeiler-Leman (WL) algorithm – cont'd

(WL algorithm)

Output “YES” if G and H have the same count of each color.

Output “NO” otherwise.

For “NO”, G and H are not isomorphic.

For “YES”, G and H are *not* necessarily isomorphic.

Weisfeiler-Leman (WL) algorithm – cont'd

(WL algorithm)

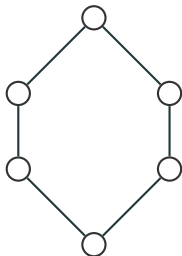
Output “YES” if G and H have the same count of each color.

Output “NO” otherwise.

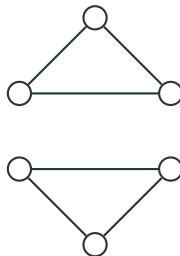
For “NO”, G and H are not isomorphic.

For “YES”, G and H are *not* necessarily isomorphic.

G :



H :



Comparison with local GNN

(Recall) On each level, it updates the feature vector of each node:

$$\bar{u} \mapsto \text{COMBINE}(\bar{u}, \text{AGG}\{\text{features of the neighbors}\})$$

The input is already a colored graph.

Comparison with local GNN

(Recall) On each level, it updates the feature vector of each node:

$$\bar{u} \mapsto \text{COMBINE}(\bar{u}, \text{AGG}\{\text{features of the neighbors}\})$$

The input is already a colored graph.

Theorem [Xu, et. al. 2019]

Let $\mathcal{A}$ be a GNN (for graph classification). For two non-isomorphic graphs G and H : if $\mathcal{A}$ outputs different features for G and H , then WL algorithm also decides that they are non-isomorphic.

Comparison with local GNN

(Recall) On each level, it updates the feature vector of each node:

$$\bar{u} \mapsto \text{COMBINE}(\bar{u}, \text{AGG}\{\text{features of the neighbors}\})$$

The input is already a colored graph.

Theorem [Xu, et. al. 2019]

Let $\mathcal{A}$ be a GNN (for graph classification). For two non-isomorphic graphs G and H : if $\mathcal{A}$ outputs different features for G and H , then WL algorithm also decides that they are non-isomorphic.

GNN $\leq$ WL:

- GNN is local, like WL.
- GNN has fixed layers, but WL has unbounded number of layers.

The bound is tight

Theorem [Xu, et. al. 2019]

There is a GNN architecture (GIN) whose separating power equals that of WL: if WL distinguishes G and H after L rounds, then some L -layer GNN distinguishes them.

The bound is tight

Theorem [Xu, et. al. 2019]

There is a GNN architecture (GIN) whose separating power equals that of WL: if WL distinguishes G and H after L rounds, then some L -layer GNN distinguishes them.

The key requirement: COMBINE and AGGREGATE must be *injective* on multisets.

- **sum** aggregation: injective on multisets.
- **max** and **mean**: *not* injective, hence strictly weaker.

The bound is tight

Theorem [Xu, et. al. 2019]

There is a GNN architecture (GIN) whose separating power equals that of WL: if WL distinguishes G and H after L rounds, then some L -layer GNN distinguishes them.

The key requirement: COMBINE and AGGREGATE must be *injective* on multisets.

- **sum** aggregation: injective on multisets.
- **max** and **mean**: *not* injective, hence strictly weaker.

Theorem [Cai, Furer, Immerman, 1992]

WL algorithm has the same separating power as the 2-variable counting logic C^2 .

- G and H are indistinguishable by WL.
- G and H satisfy the same set of C^2 sentences.

Table of contents

1. A brief review of GNNs
2. GNN and the WL algorithm
3. Uniform expressiveness: the logic C^2
4. Presburger quantifiers and the logic MP^2
5. GNNs with bounded activations: exact characterisation
6. GNNs with unbounded activations
7. Summary

The two-variable logic (C^2)

An n -color graph is $G = (V, E, U_1, \dots, U_n)$ where $U_1, \dots, U_n \subseteq V$.

(Def.) The logic C^2 :

$$\begin{aligned} \varphi := & U_i(x) \mid U_i(y) \mid E(x, y) \mid E(y, x) \mid \neg\varphi \mid \varphi \wedge \varphi \mid \\ & \forall x \varphi \mid \forall y \varphi \mid \exists^{\geq k} x \varphi \mid \exists^{\geq k} y \varphi \end{aligned}$$

Comparison with GNN

Theorem [Barcelo, et. al. 2019]

For every C^2 formula $\varphi(x)$, there is GNN $\mathcal{A}$ (with global readout) such that for every graph G and for every vertex a in G :

G, a satisfies $\varphi(x)$ iff it is accepted by $\mathcal{A}$

Comparison with GNN

Theorem [Barcelo, et. al. 2019]

For every $\mathcal{C}^2$ formula $\varphi(x)$, there is GNN $\mathcal{A}$ (with global readout) such that for every graph G and for every vertex a in G :

G, a satisfies $\varphi(x)$ iff it is accepted by $\mathcal{A}$

(Recall) GNN with global readout:

$$\bar{u} \mapsto f(A\bar{u} + B \sum_{\text{all vertices}} \bar{v} + C \sum_{\text{neighbour}} \bar{v} + d)$$

Comparison with GNN

Theorem [Barcelo, et. al. 2019]

For every $\mathcal{C}^2$ formula $\varphi(x)$, there is GNN $\mathcal{A}$ (with global readout) such that for every graph G and for every vertex a in G :

G, a satisfies $\varphi(x)$ iff it is accepted by $\mathcal{A}$

(Recall) GNN with global readout:

$$\bar{u} \mapsto f(A\bar{u} + B \sum_{\text{all vertices}} \bar{v} + C \sum_{\text{neighbour}} \bar{v} + d)$$

(Intuition)

- $\forall x$ is captured by $B \sum_{\text{all vertices}} \bar{v}$.
- $\exists^{\geq k} x$ is captured by $B \sum_{\text{all nodes}} \bar{v}$ or $C \sum_{\text{neighbour}} \bar{v}$.
- $n \geq k$ iff $1 - \text{ReLU}(k - n) = 1$.

The local case: graded modal logic

(Def.) *Graded modal logic* (GML) is the one-free-variable fragment where every quantifier is guarded by an edge:

$$\varphi(x) := U_i(x) \mid \neg\varphi(x) \mid \varphi(x) \wedge \varphi(x) \mid \exists^{\geq k} y (E(x, y) \wedge \varphi(y))$$

GML = the “local” fragment of C^2 : no global quantification.

The local case: graded modal logic

(Def.) *Graded modal logic* (GML) is the one-free-variable fragment where every quantifier is guarded by an edge:

$$\varphi(x) := U_i(x) \mid \neg\varphi(x) \mid \varphi(x) \wedge \varphi(x) \mid \exists^{\geq k} y (E(x, y) \wedge \varphi(y))$$

GML = the “local” fragment of C^2 : no global quantification.

Theorem [Barcelo, et. al. 2020]

For a node classifier $\varphi(x)$ definable in first-order logic, the following are equivalent:

- φ is expressible by a local GNN (with trReLU);
- φ is expressible in GML.

The local case: graded modal logic

(Def.) *Graded modal logic* (GML) is the one-free-variable fragment where every quantifier is guarded by an edge:

$$\varphi(x) := U_i(x) \mid \neg\varphi(x) \mid \varphi(x) \wedge \varphi(x) \mid \exists^{\geq k} y (E(x, y) \wedge \varphi(y))$$

GML = the “local” fragment of C^2 : no global quantification.

Theorem [Barcelo, et. al. 2020]

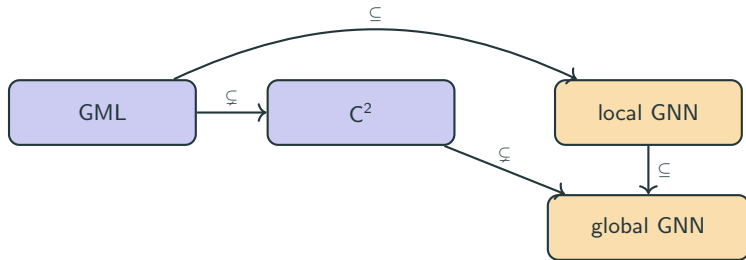
For a node classifier $\varphi(x)$ definable in first-order logic, the following are equivalent:

- φ is expressible by a local GNN (with trReLU);
- φ is expressible in GML.

(Two crucial caveats)

- The characterisation holds only *within* first-order logic.
- The C^2 result is only an *inclusion*, not an equivalence.

What we have so far



(What we don't have) Decision procedures for GNN.

C^2 is weaker than GNN

GNN can express a node property:

$$\#(\text{neighbors with color } U_1) \geq \#(\text{neighbors with color } U_2)$$

C^2 is weaker than GNN

GNN can express a node property:

$$\#(\text{neighbors with color } U_1) \geq \#(\text{neighbors with color } U_2)$$

It cannot be expressed in C^2 , because k in the quantifier $\exists^{\geq k} x$ is fixed.

C^2 is weaker than GNN

GNN can express a node property:

$$\#(\text{neighbors with color } U_1) \geq \#(\text{neighbors with color } U_2)$$

It cannot be expressed in C^2 , because k in the quantifier $\exists^{\geq k} x$ is fixed.

(Remark) GNN can do it because we can do affine transformation in COMBINE.

We need a logic that captures affine transformations.

Table of contents

1. A brief review of GNNs
2. GNN and the WL algorithm
3. Uniform expressiveness: the logic C^2
4. Presburger quantifiers and the logic MP^2
5. GNNs with bounded activations: exact characterisation
6. GNNs with unbounded activations
7. Summary

Extension with Presburger arithmetic

(Def.) A *Presburger quantifier* is of the form:

$$\mathcal{P}(x) := \sum_{t=1}^k \lambda_t \cdot \#_y[\varphi_t(x, y)] \circledast \delta$$

where $\delta, \lambda_t \in \mathbb{Z}$; each $\varphi_t(x, y)$ is a formula with free variables x and y ; $\circledast$ is one of $=, \neq, \leq, \geq, <, >$.

(Semantics) $G, v \models \mathcal{P}(x)$ if the inequality holds when $\#_y[\varphi_t(x, y)]$ is interpreted as:

$$|\{u \in V \mid G \models \varphi_t(v, u)\}|$$

Extension with Presburger arithmetic

(Def.) A *Presburger quantifier* is of the form:

$$\mathcal{P}(x) := \sum_{t=1}^k \lambda_t \cdot \#_y[\varphi_t(x, y)] \circledast \delta$$

where $\delta, \lambda_t \in \mathbb{Z}$; each $\varphi_t(x, y)$ is a formula with free variables x and y ; $\circledast$ is one of $=, \neq, \leq, \geq, <, >$.

(Semantics) $G, v \models \mathcal{P}(x)$ if the inequality holds when $\#_y[\varphi_t(x, y)]$ is interpreted as:

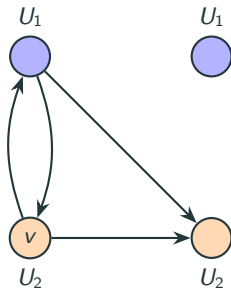
$$|\{u \in V \mid G \models \varphi_t(v, u)\}|$$

(Remark) $\exists^{\geq k} y \varphi(x, y)$ is a special case of Presburger quantification:

$$\#_y[\varphi(x, y)] \geq k.$$

Presburger quantifiers – Example

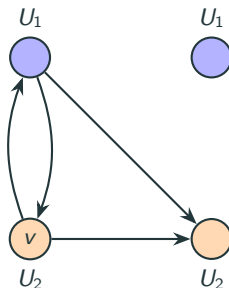
$$\mathcal{P}(x) := (-2 \cdot \#_y[E(x, y) \wedge U_2(y)] + 3 \cdot \#_y[U_1(y)] > 0)$$



G, v satisfies $\mathcal{P}(x)$: $-2 \cdot 1 + 3 \cdot 2 = 4 > 0$.

Presburger quantifiers – Example

$$\mathcal{P}(x) := (-2 \cdot \#_y[E(x, y) \wedge U_2(y)] + 3 \cdot \#_y[U_1(y)] > 0)$$



G, v satisfies $\mathcal{P}(x)$: $-2 \cdot 1 + 3 \cdot 2 = 4 > 0$.

(Note) $\#_y[U_1(y)]$ is an *unguarded* count — it looks at the whole graph.

Two-variable logic with Presburger quantifiers

(Def.)

P^2 = two-variable logic with Presburger quantifiers

GP^2 = guarded fragment of P^2 – the counting parts are of the form:

$$\#_y[E(x, y) \wedge \varphi(x, y)]$$

$\#_y[\varphi(y)]$ is not allowed.

Two-variable logic with Presburger quantifiers

(Def.)

P^2 = two-variable logic with Presburger quantifiers

GP^2 = guarded fragment of P^2 – the counting parts are of the form:

$$\#_y[E(x, y) \wedge \varphi(x, y)]$$

$\#_y[\varphi(y)]$ is not allowed.

Theorem [Bednarczyk, et. al., 2021]

- *The finite satisfiability problem of P^2 is undecidable.*
- *The finite satisfiability problem of GP^2 is decidable.*

Modal logics for GNN characterisation

(Def.) Formulas of *two-variable modal logic with Presburger quantifiers* (MP^2) will have exactly one free variable, denoted x below:

$$\begin{aligned} \varphi(x) := & \top \mid U_i(x) \mid \neg\varphi(x) \mid \varphi(x) \wedge \varphi(x) \\ & \mid \sum_{t=1}^k \lambda_t \cdot \#_y[\epsilon_t(x, y) \wedge \varphi_t(y)] \circledast \delta \end{aligned}$$

each $\epsilon_t(x, y)$ is either $E(x, y)$, $E(y, x)$ or $\top$.

(Def.) *Local* MP^2 ($\mathcal{L}\text{-MP}^2$) is obtained by excluding $\top$ as a guard atom.

(Def.) *Outgoing-only* MP^2 ($\mathcal{O}\text{-MP}^2$) excludes $E(y, x)$ as a guard atom.

Modal logics for GNN characterisation

(Def.) Formulas of *two-variable modal logic with Presburger quantifiers* (MP^2) will have exactly one free variable, denoted x below:

$$\begin{aligned} \varphi(x) := & \top \mid U_i(x) \mid \neg\varphi(x) \mid \varphi(x) \wedge \varphi(x) \\ & \mid \sum_{t=1}^k \lambda_t \cdot \#_y[\epsilon_t(x, y) \wedge \varphi_t(y)] \circledast \delta \end{aligned}$$

each $\epsilon_t(x, y)$ is either $E(x, y)$, $E(y, x)$ or $\top$.

(Def.) *Local* MP^2 ($\mathcal{L}\text{-MP}^2$) is obtained by excluding $\top$ as a guard atom.

(Def.) *Outgoing-only* MP^2 ($\mathcal{O}\text{-MP}^2$) excludes $E(y, x)$ as a guard atom.

In MP^2 the two variables appear *only in the guards* $\epsilon_t(x, y)$. Inside the count we may only put a *one-variable* formula $\varphi_t(y)$. So $\mathcal{L}\text{-MP}^2 \subsetneq \text{GP}^2$.

Modal logics for GNN characterisation

(Def.) Formulas of *two-variable modal logic with Presburger quantifiers* (MP^2) will have exactly one free variable, denoted x below:

$$\begin{aligned} \varphi(x) := & \top \mid U_i(x) \mid \neg\varphi(x) \mid \varphi(x) \wedge \varphi(x) \\ & \mid \sum_{t=1}^k \lambda_t \cdot \#_y[\epsilon_t(x, y) \wedge \varphi_t(y)] \circledast \delta \end{aligned}$$

each $\epsilon_t(x, y)$ is either $E(x, y)$, $E(y, x)$ or $\top$.

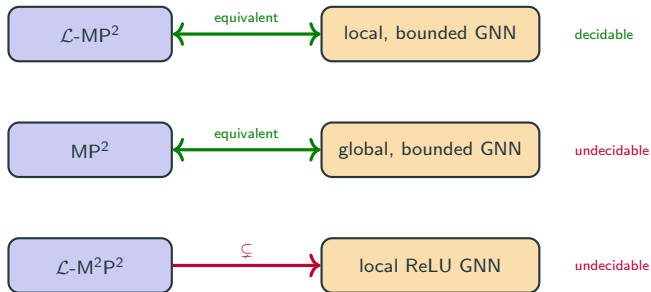
(Def.) *Local* MP^2 ($\mathcal{L}\text{-MP}^2$) is obtained by excluding $\top$ as a guard atom.

(Def.) *Outgoing-only* MP^2 ($\mathcal{O}\text{-MP}^2$) excludes $E(y, x)$ as a guard atom.

In MP^2 the two variables appear *only in the guards* $\epsilon_t(x, y)$. Inside the count we may only put a *one-variable* formula $\varphi_t(y)$. So $\mathcal{L}\text{-MP}^2 \subsetneq \text{GP}^2$.

(Def.) *Local* M^kP^2 ($\mathcal{L}\text{-M}^k\text{P}^2$) is obtained by extending $\mathcal{O}\text{-MP}^2$ with counting up the number of nodes of distance up to k .

Where we are heading



Three things to establish, in this order:

1. bounded GNN $\rightarrow$ MP^2 (via the *spectrum*);
2. $\text{MP}^2 \rightarrow$ bounded GNN (induction on formulas);
3. MP^2 is undecidable (Hilbert's tenth problem).

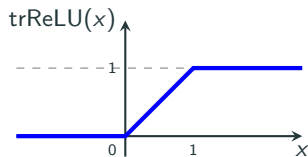
Table of contents

1. A brief review of GNNs
2. GNN and the WL algorithm
3. Uniform expressiveness: the logic C^2
4. Presburger quantifiers and the logic MP^2
5. GNNs with bounded activations: exact characterisation
6. GNNs with unbounded activations
7. Summary

Bounded GNN

Bounded GNN = GNN with trReLU as the activation function:

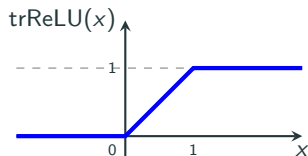
$$\bullet \text{trReLU}(x) = \begin{cases} 0, & \text{if } x \leq 0 \\ x, & \text{if } 0 < x < 1 \\ 1, & \text{if } x \geq 1 \end{cases}$$



Bounded GNN

Bounded GNN = GNN with trReLU as the activation function:

$$\bullet \text{trReLU}(x) = \begin{cases} 0, & \text{if } x \leq 0 \\ x, & \text{if } 0 < x < 1 \\ 1, & \text{if } x \geq 1 \end{cases}$$



The results hold for *eventually constant* functions.

The key object: the spectrum

(Def.) For a GNN $\mathcal{A}$ and $0 \leq \ell \leq L$, the *ℓ -spectrum* of $\mathcal{A}$ is

$$\mathbb{S}^{(\ell)} := \{ \xi_G^{(\ell)}(v) \mid G \text{ an } n\text{-graph, } v \text{ a node of } G \}.$$

The set of all features at layer ℓ (over *all* inputs).

The key object: the spectrum

(Def.) For a GNN $\mathcal{A}$ and $0 \leq \ell \leq L$, the *ℓ -spectrum* of $\mathcal{A}$ is

$$\mathbb{S}^{(\ell)} := \{ \xi_G^{(\ell)}(v) \mid G \text{ an } n\text{-graph, } v \text{ a node of } G \}.$$

The set of all features at layer ℓ (over *all* inputs).

G ranges over infinitely many graphs of unbounded size.

A priori $\mathbb{S}^{(\ell)}$ could be infinite.

The key object: the spectrum

(Def.) For a GNN $\mathcal{A}$ and $0 \leq \ell \leq L$, the ℓ -spectrum of $\mathcal{A}$ is

$$\mathbb{S}^{(\ell)} := \{ \xi_G^{(\ell)}(v) \mid G \text{ an } n\text{-graph, } v \text{ a node of } G \}.$$

The set of all features at layer ℓ (over *all* inputs).

G ranges over infinitely many graphs of unbounded size.

A priori $\mathbb{S}^{(\ell)}$ could be infinite.

Theorem (Benedikt, Lu, Motik, T. 2024)

For every bounded GNN $\mathcal{A}$ and every $0 \leq \ell \leq L$, the ℓ -spectrum of $\mathcal{A}$ is finite.

Moreover, we can effectively compute a finite superset of it (from the description of $\mathcal{A}$).

From GNN to logic

Let $\uparrow\mathbb{S}^{(\ell)}$ be the finite superset of the ℓ -spectrum.

Lemma

For every n - $\mathcal{C}$ -GNN $\mathcal{A}$, every $0 \leq \ell \leq L$, and every $s \in \uparrow\mathbb{S}^{(\ell)}$, we can compute an n -MP² formula $\varphi_s^{(\ell)}(x)$ such that for every n -graph G and node v :

$$G \models \varphi_s^{(\ell)}(v) \quad \text{iff} \quad \xi^{(\ell)}(v) = s.$$

If $\mathcal{A}$ is local (outgoing-only), then $\varphi_s^{(\ell)}$ is in $\mathcal{L}$ -MP² ($\mathcal{O}$ -MP²).

From GNN to logic

Let $\uparrow\mathbb{S}^{(\ell)}$ be the finite superset of the ℓ -spectrum.

Lemma

For every n - $\mathcal{C}$ -GNN $\mathcal{A}$, every $0 \leq \ell \leq L$, and every $s \in \uparrow\mathbb{S}^{(\ell)}$, we can compute an n -MP² formula $\varphi_s^{(\ell)}(x)$ such that for every n -graph G and node v :

$$G \models \varphi_s^{(\ell)}(v) \quad \text{iff} \quad \xi^{(\ell)}(v) = s.$$

If $\mathcal{A}$ is local (outgoing-only), then $\varphi_s^{(\ell)}$ is in $\mathcal{L}$ -MP² ($\mathcal{O}$ -MP²).

Theorem (Benedikt, Lu, Motik, T. 2024)

For every bounded GNN $\mathcal{A}$ there is an MP² formula $\psi_{\mathcal{A}}(x)$, effectively computable from $\mathcal{A}$, such that $\mathcal{A}$ and $\psi_{\mathcal{A}}(x)$ are equivalent.

For bounded local and outgoing GNN the formulas obtained are in $\mathcal{L}$ -MP² and $\mathcal{O}$ -MP² respectively.

The converse: from logic to GNN

Theorem (Benedikt, Lu, Motik, T. 2024)

For every MP^2 formula $\psi(x)$ we can compute a bounded $\mathcal{A}_\psi$ equivalent to $\psi(x)$, in time polynomial in $|\psi|$.

$\mathcal{L}\text{-MP}^2$ and $\mathcal{O}\text{-MP}^2$ formulas give local and outgoing bounded GNN, resp.

The converse: from logic to GNN

Theorem (Benedikt, Lu, Motik, T. 2024)

For every MP^2 formula $\psi(x)$ we can compute a bounded $\mathcal{A}_\psi$ equivalent to $\psi(x)$, in time polynomial in $|\psi|$.

$\mathcal{L}\text{-MP}^2$ and $\mathcal{O}\text{-MP}^2$ formulas give local and outgoing bounded GNN, resp.

This direction is *polynomial time* — unlike the other one.

Exact equivalence

Corollary

- MP^2 and bounded GNNs are expressively equivalent.
- $\mathcal{L}\text{-MP}^2$ and local bounded GNNs are expressively equivalent.
- $\mathcal{O}\text{-MP}^2$ and outgoing-only bounded GNNs are expressively equivalent.

Exact equivalence

Corollary

- MP^2 and bounded GNNs are expressively equivalent.
- $\mathcal{L}\text{-MP}^2$ and local bounded GNNs are expressively equivalent.
- $\mathcal{O}\text{-MP}^2$ and outgoing-only bounded GNNs are expressively equivalent.

(Contrast with [Barcelo et al. 2020])

Barceló et al.	this corollary
$\text{FO} \cap \text{local GNN} = \text{GML}$	local bounded GNN = $\mathcal{L}\text{-MP}^2$
characterisation <i>inside</i> FO	general characterisation
$\text{C}^2 \subseteq \text{global GNN}$	bounded GNN = MP^2
undirected only	directed, both directions

Decidability for local GNNs

Theorem [Benedikt, Lu, Motik, T. 2024]

- *The satisfiability problem of local bounded GNNs is decidable.*
- *The universal satisfiability problem of local bounded GNNs is decidable.*

Follow from the decidability of $\mathcal{L}\text{-MP}^2$ and $\mathcal{O}\text{-MP}^2$.

Decidability for local GNNs

Theorem [Benedikt, Lu, Motik, T. 2024]

- *The satisfiability problem of local bounded GNNs is decidable.*
- *The universal satisfiability problem of local bounded GNNs is decidable.*

Follow from the decidability of $\mathcal{L}\text{-MP}^2$ and $\mathcal{O}\text{-MP}^2$.

Theorem [Benedikt, Lu, Motik, T. 2024]

The finite satisfiability problem of MP^2 is undecidable.

Reduction from Hilbert's tenth problem, following [Bednarczyk et al. 2021].

Decidability for local GNNs

Theorem [Benedikt, Lu, Motik, T. 2024]

- *The satisfiability problem of local bounded GNNs is decidable.*
- *The universal satisfiability problem of local bounded GNNs is decidable.*

Follow from the decidability of $\mathcal{L}$ -MP² and $\mathcal{O}$ -MP².

Theorem [Benedikt, Lu, Motik, T. 2024]

The finite satisfiability problem of MP² is undecidable.

Reduction from Hilbert's tenth problem, following [Bednarczyk et al. 2021].

Theorem [Benedikt, Lu, Motik, T. 2024]

The satisfiability and universal satisfiability problem of bounded GNNs is undecidable.

Tight complexity: local truncated ReLU

Theorem [Benedikt, Lu, Motik, T. 2024]

The satisfiability problem of local bounded GNNs is PSPACE-complete, and so is outgoing local bounded GNN.

They are NP-complete when the number of layers is fixed.

Guess the model of each layer and check that it is consistent with the next layer.

The feature of each node takes at most polynomially many bits.

Tight complexity: local truncated ReLU

Theorem [Benedikt, Lu, Motik, T. 2024]

The satisfiability problem of local bounded GNNs is PSPACE-complete, and so is outgoing local bounded GNN.

They are NP-complete when the number of layers is fixed.

Guess the model of each layer and check that it is consistent with the next layer.

The feature of each node takes at most polynomially many bits.

Theorem [Nunn, Sälzer, Schwarzentruher, Troquard, 2024]

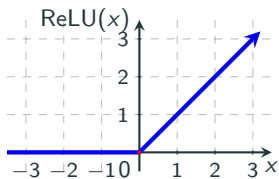
PSPACE-completeness for trReLU GNNs with integer coefficients.

Table of contents

1. A brief review of GNNs
2. GNN and the WL algorithm
3. Uniform expressiveness: the logic C^2
4. Presburger quantifiers and the logic MP^2
5. GNNs with bounded activations: exact characterisation
- 6. GNNs with unbounded activations**
7. Summary

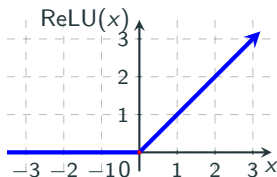
Unbounded activations

We focus on unbounded activation functions: ReLU.



Unbounded activations

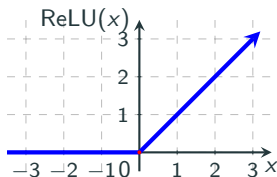
We focus on unbounded activation functions: ReLU.



(Remark) The spectrum is no longer finite. So all proof techniques in the previous section does not work.

Unbounded activations

We focus on unbounded activation functions: ReLU.



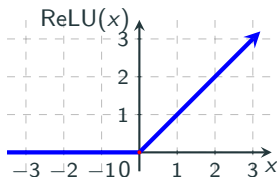
(Remark) The spectrum is no longer finite. So all proof techniques in the previous section does not work.

(Two questions)

- Do ReLU GNNs actually express *more* than bounded ones?
- Is anything still decidable?

Unbounded activations

We focus on unbounded activation functions: ReLU.



(Remark) The spectrum is no longer finite. So all proof techniques in the previous section does not work.

(Two questions)

- Do ReLU GNNs actually express *more* than bounded ones?
- Is anything still decidable?

Yes to both, but only in the “modal” corner.

ReLU GNN is more expressive than trReLU GNN

(E.g.) ReLU can count the number of nodes of distance 2.

ReLU GNN is more expressive than trReLU GNN

(E.g.) ReLU can count the number of nodes of distance 2.

Theorem [Benedikt, Lu, Motik, T. 2024]

$\mathcal{L}\text{-M}^2\text{P}^2$ is strictly more expressive than local bounded GNNs.

ReLU GNN is more expressive than trReLU GNN

(E.g.) ReLU can count the number of nodes of distance 2.

Theorem [Benedikt, Lu, Motik, T. 2024]

$\mathcal{L}\text{-M}^2\text{P}^2$ is strictly more expressive than local bounded GNNs.

Theorem [Benedikt, Lu, Motik, T. 2024]

The satisfiability and universal satisfiability problem for outgoing ReLU-GNNs is undecidable, and so is the satisfiability problem for local ReLU-GNNs.

With trReLU they are decidable.

ReLU GNN is more expressive than trReLU GNN

(E.g.) ReLU can count the number of nodes of distance 2.

Theorem [Benedikt, Lu, Motik, T. 2024]

$\mathcal{L}\text{-M}^2\text{P}^2$ is strictly more expressive than local bounded GNNs.

Theorem [Benedikt, Lu, Motik, T. 2024]

The satisfiability and universal satisfiability problem for outgoing ReLU-GNNs is undecidable, and so is the satisfiability problem for local ReLU-GNNs.

With trReLU they are decidable.

Everything with ReLU is undecidable except the “modal” corner: outgoing-only and local.

Theorem [Benedikt, Lu, Motik, T. 2024]

The satisfiability problem of outgoing local ReLU-GNNs is NEXP-complete, and NP-complete when the number of layers is fixed.

Activation functions can be any piecewise linear function.

ReLU GNN is more expressive than trReLU GNN

(E.g.) ReLU can count the number of nodes of distance 2.

Theorem [Benedikt, Lu, Motik, T. 2024]

$\mathcal{L}\text{-M}^2\text{P}^2$ is strictly more expressive than local bounded GNNs.

Theorem [Benedikt, Lu, Motik, T. 2024]

The satisfiability and universal satisfiability problem for outgoing ReLU-GNNs is undecidable, and so is the satisfiability problem for local ReLU-GNNs.

With trReLU they are decidable.

Everything with ReLU is undecidable except the “modal” corner: outgoing-only and local.

Theorem [Benedikt, Lu, Motik, T. 2024]

The satisfiability problem of outgoing local ReLU-GNNs is NEXP-complete, and NP-complete when the number of layers is fixed.

Activation functions can be any piecewise linear function.

Table of contents

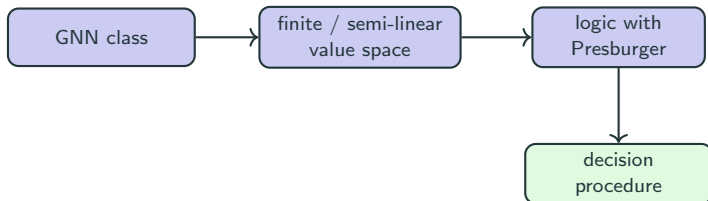
1. A brief review of GNNs
2. GNN and the WL algorithm
3. Uniform expressiveness: the logic C^2
4. Presburger quantifiers and the logic MP^2
5. GNNs with bounded activations: exact characterisation
6. GNNs with unbounded activations
7. Summary

Summary: satisfiability

	Local ($\mathcal{L}$)		Global	
	Outgoing-only ($\mathcal{O}$)	Bidirectional	Outgoing-only ($\mathcal{O}$)	Bidirectional
trReLU	PSPACE-complete		Open	Undecidable
Eventually constant ($\mathcal{C}$)	Decidable			
ReLU	NEXP-complete	Undecidable	Undecidable	
Piecewise linear (PW)				

Summary for the decidability results

(What worked)

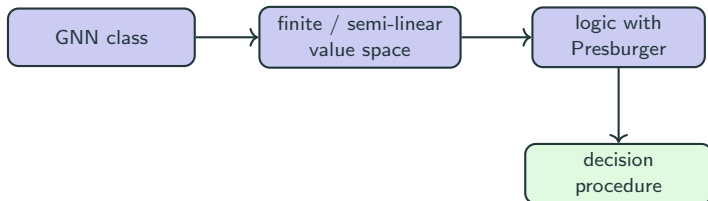


Activation functions:

- *Bounded activations.*
- *Unbounded, modal.*

Summary for the decidability results

(What worked)



Activation functions:

- *Bounded activations.*
- *Unbounded, modal.*

(Take-away) GNNs for colored graphs are not first-order logic, but closer to *guarded quantification plus arithmetic on counts*.

References

- Keyulu Xu, Weihua Hu, Jure Leskovec, Stefanie Jegelka:
How Powerful are Graph Neural Networks?
ICLR 2019.
- Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, Martin Grohe:
Weisfeiler and Leman Go Neural: Higher-order Graph Neural Networks
AAAI 2019.
- Boris Weisfeiler and Andrei Leman:
A Reduction of a Graph to a Canonical Form and an Algebra Arising during This Reduction
Nauchno-Tekhnicheskaya Informatsiya, Series 2, Vol. 9, pages 12–16, 1968.
- Jin-Yi Cai, Martin Fürer, and Neil Immerman:
An optimal lower bound on the number of variables for graph isomorphism
Combinatorica 1992.
- Pablo Barceló, Egor V. Kostylev, Mikaël Monet, Jorge Pérez, Juan Reutter, Juan-Pablo Silva:
The Logical Expressiveness of Graph Neural Networks
ICLR 2020.
- Martin Grohe:
The Logic of Graph Neural Networks (LICS 2021); The Descriptive Complexity of Graph Neural Networks (LICS 2023).
- Timon Barlag, Vivian Holzapfel, Laura Strieker, Jonni Virtama, Heribert Vollmer:
Graph Neural Networks and Arithmetic Circuits
NeurIPS 2024.

References – verification and decidable logics

- Michael Benedikt, Chia-Hsuan Lu, Tony Tan:
Decidability of Graph Neural Networks via Logical Characterizations.
ACM Trans. Comput. Log. 27(2): 13:1-13:65 (2026).
Conference version: Benedikt, Lu, Motik, Tan, ICALP 2024: 127:1–127:20.
- Michael Benedikt, Chia-Hsuan Lu, Tony Tan:
Analysis of Logics with Arithmetic.
CSL 2026: 27:1-27:18.
- Chia-Hsuan Lu, Tony Tan, Michael Benedikt:
Robustness Verification of Graph Neural Networks Via Lightweight Satisfiability Testing.
TACAS (1) 2026: 3-22.
- Chia-Hsuan Lu, Tony Tan:
On two-variable guarded fragment logic with expressive local Presburger constraints.
Log. Methods Comput. Sci. 20(3) (2024).

References – verification and decidable logics (contd.)

- Bartosz Bednarczyk, Maja Orlowska, Anna Pacanowska, Tony Tan:
On Classical Decidable Logics Extended with Percentage Quantifiers and Arithmetics.
FSTTCS 2021: 36:1-36:15.
- Pierre Nunn, Marco Sälzer, François Schwarzentruher, Nicolas Troquard:
A Logic for Reasoning About Aggregate-Combine Graph Neural Networks
IJCAI 2024.
- Marco Sälzer, Martin Lange:
Fundamental Limits in Formal Verification of Message-Passing Neural Networks
ICLR 2023.
- Christoph Haase, Georg Zetsche:
Presburger Arithmetic with Stars, Rational Subsets of Graph Groups, and Nested Zero Tests
LICS 2019.
- Christopher Hojny, Shiqiang Zhang, Juan S. Campos, Ruth Misener:
Verifying message-passing neural networks via topology-based bounds tightening
ICML 2024.
- Minghao Liu, Chia-Hsuan Lu, Marta Kwiatkowska:
Exact Verification of Graph Neural Networks with Incremental Constraint Solving.
FM (1) 2026: 641-662.

The predecessors of this work – the logic side



Thank you! Next: Michael's talk